1.1 订单簿的合成逻辑

订单簿引擎采用基于消减的合成逻辑，仅需输入逐笔委托与逐笔成交数据即可：

- 在一个通道内，按 ApplSeqNum 有序逐条处理逐笔数据。ApplSeqNum 为交易所原始消息记录号，也称为 BizIndex，同频道内自 1 开始计数；
- 当收到委托时缓存委托单的价格和量，当收到成交和撤单时，从缓存中减去对应委托单的价格和量；
- 通过引擎内部缓存的委托单信息可以计算出每个报价的委托量、委托笔数等信息。

整体的合成逻辑如上，在此基础上也会遵循交易规则修正后再输出订单簿，以保证输出结果的准确性。比如对于深交所股票，引擎支持创业板价格笼子判断，部分报价会因为触发价格笼子规则而不体现在订单簿中，用户可以通过参数控制是否启动该规则。

通过将引擎计算结果与交易所发布的 3 秒快照进行比较，可以验证引擎内部合成逻辑的正确性。具体的校验方案与实现脚本见本文第 5 章。

1.2 对时间窗口的处理

订单簿快照表示了某个时刻的订单簿截面，截面表示的窗口范围以及何时进行截面都由引擎自动控制。接下来我们介绍引擎在窗口时间、触发窗口的时机等方面的工作原理。

当订单簿引擎设置的输出频率为 1 秒时，若输出一条时间戳字段为 10:00:00 的计算结果，该计算结果表示的是时间戳小于等于 10:00:00 的全部逐笔形成的最新订单簿。一些窗口指标如当前周期内成交量，则对应的是在 (09:59:59, 10:00:00] 这个左开右闭区间内的逐笔数据的总成交量。仅有时间戳为 14:57:00 的结果的含义稍有不同，引擎的输出到 14:57:00 就会停止，即引擎的处理与输出不包含收盘集合竞价阶段，所以 14:57:00 的计算结果对应 (14:56:59, 14:57:00) 这个左开右开区间内的逐笔数据。

在实时合成时，逐笔数据逐条进入引擎，引擎需要一个规则来判断何时窗口内的逐笔数据全部到齐了。以输出 10:00:00 的订单簿为例，判断规则是一旦收到一条时间戳大于 10:00:00 的逐笔数据（包含委托和成交），则认为应该输出 10:00:00 的计算结果了。计算结果中不包含触发输出的这条逐笔数据。此判断规则可以基于单支标的或全部标的。当指定单支标的时，每支标的收到一条时间戳大于 10:00:00 的逐笔数据时，仅触发计算并输出该标的的订单簿。当指定全部标的时，收到任意标的的时间戳大于 10:00:00 的逐笔数据时，触发计算并输出全部标的的订单簿。

此外，若某一段时间的逐笔数据空缺，则输出也会空缺，以午盘休市为例，引擎不会输出11:30:01-12:59:59 时间戳的订单簿，因为没有这段时间的逐笔数据。

严格地按照逐笔数据里的时间戳截取订单簿快照，将能够保证实盘和历史计算总能得到一致的结果，实现“流批一体”。根据已有的落地实践经验，在实盘中，我们推荐将一个通道中全部的标的视为一个整体，用全部标的的事件时间（即数据中的交易时间字段）来判断何时截取快照，这种方式能够保证尽快且准确地推出订单簿快照。此外，为了满足不同的场景需求，后续版本中也将推出基于机器时间的订单簿处理，即机器时间每经过一个时间间隔输出一次最新的订单簿。

1.3 对乱序数据的处理

在实盘环境中，接入到系统中的逐笔数据可能发生乱序，如不能正确处理乱序会影响订单簿合成的正确性。在交易所内部系统中，每个交易通道中的标的发生的委托和成交事件都会被赋予一个顺序号，即 ApplSeqNum。这个顺序号反映了事件在交易所内部实际发生的顺序。数据乱序，是指可能是由于网络延迟或其他技术问题导致的事件没有严格按照ApplSeqNum 到达数据处理系统。因此，数据处理系统收到的逐笔数据可能需要重新排序，以确保它们反映了在交易所内部实际发生的顺序。

例如，如果标的A的逐笔成交比对应的逐笔委托先到达，引擎按到达顺序处理，先处理成交再处理委托则会得到错误的订单簿结果。为此，订单簿引擎支持指定输入数据中的ApplSeqNum 字段，引擎会检测并缓存 ApplSeqNum 发跳变的数据，在接收到正确ApplSeqNum 的数据后再计算输出，可以保证订单簿合成的正确性。用户可以根据业务需求和接收到的数据质量，选择是否开启处理乱序数据的功能。

下图展示了收到不同的 ApplSeqNum 时的处理方式，从左至右表示了时间先后顺序，即最先收到ApplSeqNum=1的输入数据。

收到的 **ApplSeqNum**	**1**	**2**	**5**	**6**	**9**	**3**	**4**
此次处理的 ApplSeqNum	处理1	处理2	无处理（缓存5）	无处理（缓存5、6）	无处理（缓存5、6、9）	处理3	遍历处理4、5、6（缓存9）

1.4 统一内部数据结构

统一结构字段	sz-order	sz-trade	sh-order&trade
code	SecurityID	SecurityID	SecurityID
time	TransactTime	TransactTime	TickTime
event_type	0（委托）	ExecType 映射： 成交→2 撤销→1	Type 映射： A→0(委托) D→1(撤单) T→2(成交) S→3(状态，过滤)
price	Price	LastPx	Price
qty	OrderQty	LastQty	Qty
order_id	ApplSeqNum	根据方向选择： 买单→BidApplSeqNum 卖单→OfferApplSeqNum	根据TickBSFlag选择： B→BuyOrderNO S→SellOrderNO
counter_order_id	0	根据方向选择： 买单→OfferApplSeqNum 卖单→BidApplSeqNum	根据TickBSFlag选择： B→SellOrderNO S→BuyOrderNO
side	Side 映射： 买→1 卖→2	根据BidApplSeqNum/OfferApplSeqNum判断	TickBSFlag 映射： B→1 S→2
order_type	OrdType 映射： 市价→1 限价→2 本方最优→3	0（成交无订单类型）	0（上交所只有限价）
seq	ApplSeqNum	ApplSeqNum	BizIndex